Computational Materials Science 253 (2025) 113852

Contents lists available at ScienceDirect

Computational Materials Science

journal homepage: www.elsevier.com/locate/commatsci

Full length article

A decision transformer approach to grain boundary network optimization

Christopher W. Adair *, Oliver K. Johnson

Department of Mechanical Engineering, Brigham Young University, Provo, UT 84602, USA

ARTICLE INFO

Dataset link: A-Decision-Transformer-Approach-to-Grain-Boundary-Network-Optimization (Original data)

Keywords:
Grain boundary
Grain boundary networks
Machine learning
Decision Transformer
Human–computer interaction

ABSTRACT

As microstructure property models improve, additional information from crystallographic degrees of freedom and grain boundary networks (GBNs) can be included in microstructure design problems. However, the high-dimensional nature of including this information precludes the use of many common optimization approaches and requires less efficient methods to generate quality designs. Previous work demonstrated that human-in-the-loop optimization, instantiated as a video game, achieved high-quality, efficient solutions to these design problems. However, such data is expensive to obtain. In the present work, we show how a Decision Transformer machine learning (ML) model can be used to learn from the optimization trajectories generated by human players, and subsequently solve materials design problems. We compare the ML optimization trajectories against players and a common global optimization algorithm: simulated annealing (SA). We find that the ML model exhibits a validation accuracy of 84% against player decisions, and achieves solutions of comparable quality to SA (92%), but does so using three orders of magnitude fewer iterations. We find that the ML model generalizes in important and surprising ways, including the ability to train using a simple constitutive structure–property model and then solve microstructure design problems for a different, higher-fidelity, constitutive structure–property model without any retraining. These results demonstrate the potential of Decision Transformer models for the solution of materials design problems.

1. Introduction

Grain boundary networks (GBNs) are high-dimensional structural features in microstructures that have been useful in modeling structure–property linkages [1–10]. These models enable the simulation of grain boundary engineering, or the design of materials through manipulation of the microstructure [3,4,6,11,12]. Successes in grain boundary engineering have enabled materials to be designed with improved diffusivity [3,13–16], corrosion resistance [1,6,13], heat transfer [17–19], and mechanical properties [12,20,21].

However, the dimensionality of the GBN state space is high: $3n_{GB}$ degrees of freedom for orientations and $2n_{GB}$ degrees of freedom for the boundary normals, where n_{GB} is the number of grain boundaries, not including the additional degrees of freedom from GB connectivity [22,23]. Therefore, common design optimization methods, such as gradient ascent, are at risk of finding local, rather than global, maxima [24,25].

Global optimization methods such as simulated annealing (SA) can overcome this weakness, but come with a computational efficiency trade-off [24]. Research into other high-dimensional systems such as quantum computing and protein folding found that human input in the form of a video game can act as a simplifying heuristic on their respective spaces, and as a result, can solve optimization problems in high-dimensional spaces more effectively than stochastic methods [26–28]. Previous work on GBNs specifically found that formulating a GBN design problem as a video game led to human players generating both better and more efficient optimization pathways than stochastic methods such as SA [25].

However, obtaining human inputs in this manner is expensive in time, development, and distribution costs. It is also difficult to use high-fidelity, but computationally expensive, material models within the constraints of video game design. Therefore, learning and/or replicating generalizable human optimization strategies could provide the benefits of human decision-making.

Machine learning (ML) is the obvious approach for this because of its ability to represent very high-dimensional information, generate optimization pathways, and learn from human inputs [29–32].

Transformer-based models have received much attention recently through Generative Pre-Trained Transformer (GPT) style implementations and classification tasks [29–31,33]. Transformers function by learning the attention, or correlations, given to specific inputs, essentially creating a dictionary lookup of input to expected output [31]. The goal of this representation is a more generalizable solution that can handle variable inputs and sets of states. This dictionary can be visualized to show how much certain states, inputs, actions, or words

* Corresponding author.
E-mail address: cwa367@byu.edu (C.W. Adair).

https://doi.org/10.1016/j.commatsci.2025.113852
Received 20 November 2024; Received in revised form 21 February 2025; Accepted 16 March 2025
Available online 1 April 2025

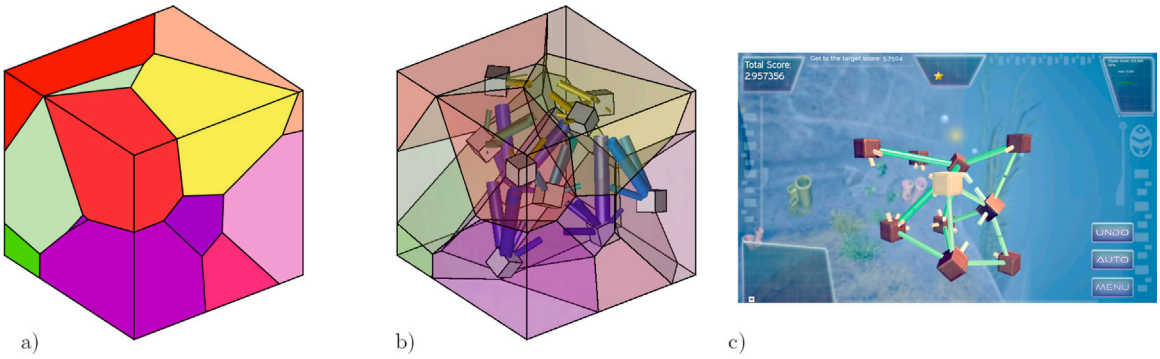

Fig. 1. A visual description of how the GBN design problem was formulated in a video game context. (a) Shows an example microstructure as generated by Neper Polycrystal [40]. In (b) the grain centers are given a cube representing the lattice orientation of the respective grain, and the connections represent the magnitude of the corresponding GB property. In (c) the microstructure mesh is hidden, showing only the cubes and connections, and the actions and properties are given UI elements to communicate the design problem to players.

correlate with each other, giving a more accessible interpretation of the high-dimensional information [31,33,34].

In this work, we generate a transformer-based ML model, and give a comparison of the solution quality and the solution efficiency of the players, the ML model, and a popular traditional global optimization algorithm (SA). We comment on the generalizability of the ML model. In particular, we explore generalization to a higher fidelity structure–property model without any additional training.

2. Background

At the mesoscopic scale, grain boundaries are often defined by average grain misorientations and plane normals [22,23]. These structural definitions have been found to correlate with multiple material properties such as diffusivity [3,13–16], corrosion [1,6,13], heat transfer [17–19], mechanical properties [12,20,21], and others [11,13]. By informing material design from these discoveries, multiple advancements have been made in corrosion resistance [1,6], battery life extension [35,36], desired diffusion effects [15,37], hydrogen embrittlement [3,16], and others [11,13,17–19,21]. Generally, these successes have been achieved by implementing methods of material synthesis that enhance the *statistical population* of targeted types of GBs [3,18,35].

Connectivity and other long-range effects of GBNs can also affect properties such as transport and fracture behavior [1,5,35,38]. Structure–property models that take these effects into account can be used to study and obtain optimal processes, states, and GB character that produce desired material performance, though design exploration of this space is necessarily complex due to the high number of interconnected degrees of freedom [7,11,23]. If exploration of this GBN design space can be done efficiently, then similar advances can be made to material design that can target *spatial configurations* of boundaries in addition to their *statistical populations*.

However, the configuration space of GBNs defined by the grain orientations and grain boundary plane normals becomes prohibitively large for conventional optimization or searching strategies such as gradient ascent, as well as having the increased risk of finding local, rather than global, maxima [24,25]. This is of particular concern for GB structure–property models, which possess sharp cusps due to crystallographic symmetries [39].

If the goal of a material design problem is searching for optimal structures within the design space, then most of the space likely consists of sub-optimal solutions that do not need extensive definition or attention, and, if skipped, could reduce the computations necessary to identify design candidates for more extensive testing [27,41,42]. Therefore, creating simplifying heuristics on GBN design optimization could enable design on not only statistical distributions, but boundary connectivity configurations as well (which requires a much larger design space).

One source for obtaining simplifying heuristics comes from human behavior, and has been studied extensively in human–computer interactions [26,43,44]. By formulating an optimization problem as a game, human inputs achieve the simplifications needed to optimize high-dimensional systems, like the GBN design problem [25,27,28,44]. Previous work specifically for GBNs found that human inputs were capable of outperforming a global optimization algorithm on the design optimization of small GBNs, especially in decision efficiency [25].

There are multiple approaches to modeling GBNs for simulation. We use a graph representation, like previous work, that gives us a model of material performance based on the full connectivity and crystallographic information needed to describe GBNs [5,25]. The materials design video game from our previous work, Operation: Forge the Deep [45], modeled the system as the dual of the GBN and allowed players to manipulate the crystallographic orientations of grains only (while keeping the geometry of the polycrystal fixed) [25]. Each crystal orientation was visually represented as an orientable cube, while each grain boundary was represented as a connection between the pair of cubes (grain orientations) incident to the GB (see Fig. 1). Players were allowed to rotate each cube either manually, or with a local, greedy gradient ascent algorithm.

The length, girth, and color of each connection provided a real-time visual representation of the corresponding grain boundary property—diffusivity in this case—based on a constitutive model that took as inputs the GB plane and the orientations of the incident grains [5,25]. A green, fully touching, large diameter connection denoted an optimal property for that boundary in the game representation (see Fig. 1c).

The players' goal was to maximize the magnitude of these connections, thereby maximizing the material property of interest [25]. The form this problem takes is then visibly recognizable as a variable size graph, where the nodes represent lattice orientations, and the edges represent grain boundary properties. This form of problem has been a popular subject for machine learning, aptly named graph-based learning [46–49].

Graph-based learning has been studied mainly in classification tasks, such as molecular identification, protein classification, and others [48–50]. However, the GBN design problem is not classification, but optimization as a sequence of decisions. Recent work has sought to tailor ML to sequence optimization through a new model called the transformer [31].

Transformer models are useful in this context because they allow for all kinds of inputs—provided there is an encoding available—and they are able to have flexible input sizes [31]. The graph nature of problems is also included intrinsically through a position encoding. The position encoding then allows for learning on different configurations. However, transformers have tended to require large amounts of data for quality training [29–31].

Decision Transformers are specific transformer models that model a time series of states, actions, and returns [30,32]. Since the human decision-making process for the current problem closely matches this (i) state (the GBN configuration), (ii) action (grain selection and orientation change), and (iii) return structure (effective material property), we can apply this model well to our gathered player input data.

Other models exist, but other studies have already compared them against each other and found that decision transformers are the current best solution for highly variable state spaces of this structure [30,32].

We modify code created for the Multi-Game Decision Transformer [30] and the Decision Transformer [32] to generate this model. We expand their models to include multiple agents, which allowed for variable sized GBN simulations. We compare the ML performance against human player performance as well as SA, a common stochastic global optimization algorithm.

We find that our modified Decision Transformer, trained on human player data, can effectively and efficiently optimize GBN properties. We detail the full model construction, validation, and evaluation and we explore ways in which the ML model generalizes.

3. Methods

We first describe the materials design problem, then we define the constitutive GB structure–property model, as well as the homogenization model to predict the effective property of the GBN as a whole. We then describe the three different optimization methods that we will compare for solution of the design problem: stochastic global optimization through SA, human inputs from a video game, and a Decision Transformer trained on human player data from the video game.

3.1. The microstructure design problem

In this study, we give the example of, and focus on, the problem of designing a microstructure to maximize the rate of hydrogen diffusion along the GBN in polycrystalline Ni in a Type-A kinetic regime [51]. The only features of the microstructure that we will use as design variables are the orientations of the respective grains, which, in turn, influence the crystallographic character of the intervening GBs. All other attributes of the microstructure (e.g. grain size and morphology, composition, etc.) and environment (e.g. temperature) will be held constant. This design task is relevant for processes such as hydrogen separation during synthesis [52], where the permeation time of hydrogen through the nickel membrane depends partially on the diffusivity. Therefore, increases in membrane diffusivity can decrease the time required for hydrogen separation without requiring higher pressures. Additionally, nickel based materials can catalyze multiple hydrogen production reactions, leading to opportunities for combining production and separation operations [52,53].

This particular design problem is given merely as an example, as our primary objective is the development of the computational method itself for application to GBN design problems generally. Indeed, the methods discussed here can easily be applied to arbitrary GBN design tasks, given suitable GB structure–property models.

Although outside the scope of the present work, it is also worth mentioning the potential for experimental synthesis of designed microstructures found via such optimization strategies using methods such as templated grain growth [54], and layer-wise engineering of grain orientations [55], which are promising experimental techniques for statistical and even spatial control of grain orientations.

In a Type-A kinetic regime, the effective diffusion coefficient of a polycrystal can be estimated using a rule of mixtures [56]:

$$D^{EFF} = (1-\eta)\, D^{XL} + \eta D^{GB} \tag{1}$$

where D^{EFF} is the effective diffusivity of the polycrystal, D^{XL} is the effective bulk crystal diffusivity, D^{GB} is the effective diffusivity of the GBs, and η is the volume fraction of GBs.

As mentioned already, the only design variables we consider are the grain orientations (which, in turn, influence the crystallographic character of the intervening GBs). Because diffusivity is a rank 2 tensor property, it is isotropic for cubic crystals [57,58], which are considered here. Because grain size is not a design variable, η is also constant. Consequently, the first term in Eq. (1) is a constant with respect to the design variables. Thus, optimization of D^{EFF} is equivalent to optimization of D^{GB} under our stated conditions. Therefore, we will only calculate D^{GB} to measure the performance of the optimization. As D^{EFF} and D^{XL} are not needed, in what follows we will use $D_{eff} \equiv D^{GB}$ to denote the effective diffusivity of the GB network, and D to denote the diffusivity of an individual GB.

The effective diffusivity of the GBN as a whole is calculated using a previously derived homogenization model which takes into account all crystallographic information and grain boundary connectivity information to predict the effective diffusivity (D_{eff}) of a 3D GBN. We summarize the key points here.

3.1.1. The GB structure–property model

A required input for the homogenization model [5] is a constitutive GB structure–property model that takes the 5 crystallographic degrees of freedom of each GB as input (3 for misorientation + 2 for plane inclination), and returns the value of the GB property of interest (in this case, the diffusivity of H in Ni GBs). For the present work, we will employ the model from Page, et al. for GB diffusivity of hydrogen in nickel [59]. This model is comprised of (i) a refinement of the Borisov relation, which predicts GB diffusivity from GB energy [60], and (ii) the Bulatov, Reed, and Kumar (BRK) model for GB energy as a function of the 5 crystallographic degrees of freedom of the GB [39]. This model was validated against molecular dynamics simulations of H diffusion in Ni GBs [59]. Advances in Scanning-TEM have also allowed for some experimental validation of the Borisov relation at low temperatures [61].

Fig. 2 shows the resulting model for H diffusivity in Ni GBs as a function of 5D GB character at a temperature of 700 K. The subplots show the GB diffusivity as a function of the GB plane normal (see annotated Miller indices) for several low-Σ misorientations. As is apparent, the variation of GB diffusivity is a complex function of the 5D GB character.

The materials design video game that provided the training data presents real-time predictions of the effective diffusivity of the GB network (the "score" that the players see) as players manipulate the microstructure. This requires the use of a constitutive model that is fast to evaluate. Unfortunately, the Borisov/BRK diffusivity model just described, while realistic, is too computationally expensive to provide the needed real-time property calculations. Instead, the training data was collected using the following simple toy model for GB diffusivity:

$$D = \beta\left(\frac{\theta}{10} + |N_x| + |N_y| + |N_z|\right) \tag{2}$$

where θ is the disorientation angle between the neighboring grains, $\mathbf{N} = \left[N_x, N_y, N_z\right]^{\mathrm{T}}$ is the GB plane unit normal in the lab frame, and $\beta = 10^7$ is an arbitrary scaling parameter. We will refer to this as the "Linear model". The form of this model is similar to test functions used for GB properties in other studies [62,63]. This simple model satisfies the computational performance requirements of the video game, while retaining dependence on some of the GB parameters and facilitating investigation of the high-dimensional GBN design optimization problem using the video game. This is purely a computational expedient. Indeed, the only similarity between the Linear and the Borisov/BRK models is that the location of the minimum occurs at a disorientation of 0°. Nevertheless, we hypothesize that the ML model can still learn useful optimization strategies from the resulting training data. We then attempt to apply the ML model, in the context of the realistic Borisov/BRK constitutive model, without retraining. This would imply that the ML model could be trained using one (simple) constitutive GB model, and then generalize to make predictions for a different (higher-fidelity) constitutive GB model. Testing this hypothesis of generalizability is one of the objectives of the present work and will be discussed later.

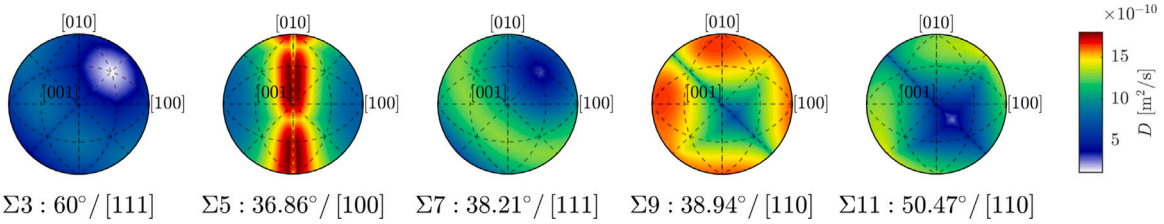

Fig. 2. Borisov/BRK diffusivity model for H in Ni GBs at 700 K. Subplots show the GB diffusivity as a function of the GB normal in the crystal reference frame (see annotated crystal directions) for several low-Σ misorientations.

3.1.2. 3D microstructure representation

The GB normals required by the constitutive GB structure–property models are obtained from a surface mesh of triangular elements along the grain boundaries of a given microstructure for a finite volume (FV) method representation [25]. Forty-six 3D microstructures were generated and meshed using Neper polycrystal [40] and given grain growth morphologies to better represent realistic samples. The generated microstructures spanned 10 different numbers of grains, ranging from 4 to 35. An example of a generated structure can be seen in Fig. 1.

To fully define a FV element, we set the grain boundary thickness at a constant 5Å, while the remaining geometric parameters of the GB are determined from the local mesh geometry.

3.1.3. Calculating effective diffusivity

The effective diffusivity of H across the entire GBN (D_{eff}) is calculated through a mass flow PDE, where mesh vertices on the boundary are given either Dirichlet or Neumann boundary conditions. We follow the same boundary conditions as our previous work [25], where the faces perpendicular to the x-direction have Dirichlet conditions for the concentrations of $c_{source} = 1\,\mathrm{kg/m^3}$ and $c_{sink} = 0\,\mathrm{kg/m^3}$ respectively, and all other directions have Neumann boundary conditions applied. For computational simplicity, all Dirichlet condition boundary vertex indices are combined into a single source (v_{source}) or sink (v_{sink}) index, respectively. This maintains the edge information of the mesh, but simplifies the PDE setup.

The Laplacian, $\boldsymbol{\mathcal{L}}$, of the GBN mesh is calculated according to:

$$\mathcal{L}_{ij} = \begin{cases} \sum_{i\sim m} \dfrac{D_{im}A_{im}}{L_{im}} & \text{if } i = j \\ -\dfrac{D_{ij}A_{ij}}{L_{ij}} & \text{if } i \sim j \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

where D_{ij} is the diffusion coefficient assigned to the mesh edge connecting vertices i and j as calculated from the chosen GB constitutive model (the Linear model or the Borisov/BRK model), A_{ij} is the element area, and L_{ij} is the element length (see [25] for detailed definitions).

Following the method described in [64], we can compute D_{eff} through the total mass flow to the sink from the source according to

$$D_{eff} = \frac{l_s}{w_s t_s c_{source}} \left(-\boldsymbol{\mathcal{L}}_{[v_{sink},\cdot]} \mathbf{Q}^{-1} \mathbf{b} \right) \tag{4}$$

where l_s, w_s, and t_s are the length, width and thickness of the material sample respectively, $\boldsymbol{\mathcal{L}}_{[v_{sink},\cdot]}$ denotes the v_{sink}th row of $\boldsymbol{\mathcal{L}}$, $\mathbf{b} = c_{source}\mathbf{e}_{source} + c_{sink}\mathbf{e}_{sink}$, and $\mathbf{Q}$ is defined as

$$\mathbf{Q} = \begin{bmatrix} \boldsymbol{\mathcal{L}}_{[F,\cdot]} \\ \mathbf{e}_{source}^{\mathsf{T}} \\ \mathbf{e}_{sink}^{\mathsf{T}} \end{bmatrix} \tag{5}$$

where F is the set of vertices excluding the source and sink (i.e. $\boldsymbol{\mathcal{L}}_{[F,\cdot]}$ is $\boldsymbol{\mathcal{L}}$ with the source and sink rows removed), with $\mathbf{e}_i$ denoting the vector whose ith element is 1 and all others are 0. For a more comprehensive explanation of the derivation we refer the reader to [64].

Given this homogenization relation, the objective of the present GBN design problem is to maximize D_{eff}, by changing the crystallographic orientations of grains in the polycrystal.

3.2. Optimization through a stochastic method

The first method we consider for optimizing D_{eff} is a stochastic global optimization algorithm called simulated annealing (SA) [24]. SA functions by taking random steps through the optimization landscape and either accepting a step that *improves* the solution, or accepting a step that *worsens* the solution with a given probability that decreases over time [24]. Accepting worse steps enables the solution to avoid local maxima better than simple gradient ascent, but comes at a computational cost. This is especially true for complex, expensive models such as the Borisov/BRK model.

For the GBN optimization, we use a Cauchy annealing schedule [65] where at each Monte Carlo step a grain is randomly selected, via uniform distribution, from the full GBN and assigned a new orientation, sampled uniformly from SO(3). We define the convergence criterion to be 1000 consecutive rejected steps. Five runs of SA using the Linear model were done for each of the evaluation microstructures, which will be defined subsequently. However, due to the complex and computationally expensive nature of the Borisov/BRK model, only one Borisov/BRK model run of SA for each evaluation structure was possible with the available computational resources.

3.3. Optimization through a video game

The second method we consider for optimizing D_{eff} is through human inputs via a video game. The game, "Operation: Forge the Deep", is a 3D puzzle game where users manipulate cubes, representing grain orientations, to change the connections, representing individual GB properties, to optimize a score, representing the effective material property of the polycrystal [45]. A screen-shot of the video game is shown in Fig. 1.

Players were allowed to perform one of the following actions to modify the orientation of a single grain at a time:

- Apply a manual rotation
- Apply a locally optimal rotation via gradient ascent
- Undo the previous action

Additionally, certain scenarios (levels) in the game antagonistically introduce sporadic random rotations to grains, meant to throw off the player's thought process and increase the challenge.

The players' goal in each scenario presented to them (i.e. each level) was to reach a score corresponding to 90% of the maximum D_{eff} solution found for that GBN morphology using SA with the Linear model. GBN morphologies were repeated in separate scenarios with additional restrictions such as: limited number of actions, limited time to solve, limited grain selection, and combinations of these restrictions. These constraints were implemented in different game "levels"

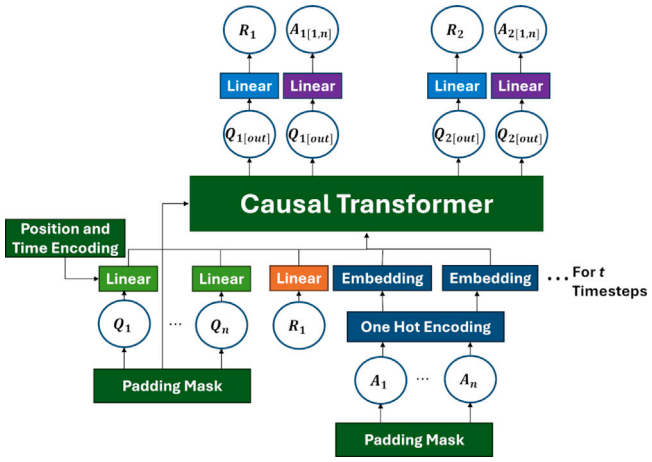

Fig. 3. Block diagram of the GBN design Decision Transformer ML model.

to encourage player engagement, but they also may encourage varied solution approaches, thereby diversifying the training set for the ML approach.

Previous studies on human inputs in this game environment were done in a laboratory setting on a small scale and with specific conditions of interest [25]. However, the current work uses data collected from a public distribution of the game through the Steam storefront, in accordance with data privacy agreements and releases [45]. Additionally, this game was presented to K-12 students as part of a STEM outreach program that also contributed to data collection. In total, 879 optimization trajectories were collected from users, which consisted of between 5 and 810 decisions per trajectory.

These trajectories were stored as a sequence of decisions, where the action, resultant state of all grain orientations, and resultant score were stored in sequence.

3.4. Optimization through a decision transformer

The third method we consider for optimizing D_{eff} is creating, training, and evaluating a Decision Transformer that uses sequences of player decisions, called player trajectories. While we base the implementation of this Decision Transformer on the original [32] and the Multi-Game Decision Transformer [30], there were substantial changes that needed to be made to apply the technique to the GBN design problem. An overview of the ML model can be seen in Fig. 3, which will be described in detail.

3.4.1. State, action, and return space

To prepare player data for training of the Decision Transformer model, the state, action, and returns must be clearly defined and quantified. The state space observed by the players includes the set of grain orientations and the magnitude of the connections between them (D_{ij} from Eq. (3)). The grain orientations were represented as a set of quaternions. The shape of the resulting state tensor is $[B \times t \times n \times 4]$, where B is the batch size, t is the number of time steps included, n is the number of grains in a simulation, and 4 represents the quaternion information (each quaternion is a 4D unit vector).

The return is simple to implement, as the game score (value of D_{eff}) represents the full return. However, normalizing the return requires a new definition. As the GBN design problem is a configurational optimization problem with many local maxima, it is unknown *a priori* if a configuration exists in which all boundaries exhibit the maximum diffusivity allowed by the underlying GB constitutive model while also satisfying all physical (e.g. crystallographic) constraints.

Therefore, while the true maximum may be unknown, a standardized normalization can be calculated by artificially setting all boundaries in a given microstructure to the maximum value of the selected GB constitutive model, and then calculating the resulting value of D_{eff} as an upper bound on the theoretical maximum. Each structure is normalized by its respective upper bound value, so all returns are scaled between 0 and 1. This encourages generalizability of training both between microstructure morphologies and possibly material models as well. The resulting input tensor is of the shape $[B \times t \times 1]$.

The action space representation differs from other Decision Transformers, since n can have a variable size (different levels correspond to polycrystals with different numbers of grains), therefore, there is a variable size in the action space as well. The action space is set as a one-hot encoding consisting of 5 possible actions for each grain:

1. Do nothing
2. Manual rotation
3. Local gradient ascent
4. Undo
5. Assign random orientation

The resulting input tensor is of the shape $[B \times t \times n \times 5]$, with the same variable meanings as the state tensor. Although only one grain is changed during a given step, the ML model technically updates the state of *all* grains at each step; thus, the "Do nothing" action is simply what is assigned to all grains that were not selected for the active action assigned at that step (actions 2–4 in the list above).

3.4.2. The input layer

Each input tensor is then projected to the hidden size, h, for the ML model. The state and return embeddings use a Linear layer each, resulting in tensor shapes of $[B \times t \times n \times h]$ and $[B \times t \times 1 \times h]$ respectively. The actions embedding uses an Embedding layer, which is useful for the sparse nature of the information, and results in a tensor shape of $[B \times t \times n \times h]$.

To account for the variable value of n, a mask is applied to the state and action tensors that prepends zeros to each tensor such that, regardless of the actual number of grains in a particular problem, the tensor will have a size of $n_{pad} = 50$ in the n dimension [66]. This mask is saved so the ML model does not take into account these zeros during training or evaluation.

With the n dimension having fixed size, the input tensors can be combined in a way that models the decision-making nature of the inputs. The inputs are interleaved according to t so that the information has the order state, return, action, for each time step t. This follows the logic of previous Decision Transformers, stating that an expected return and selected action can be predicted from an observed state, and that future actions are influenced by previous decisions [30,32].

3.4.3. The position encoding

As constructed, the input tensors do not explicitly include information about the GB properties nor the order in which decisions were made. Both kinds of information can be added through a *position encoding* [46,47,49]. Position encodings have been used especially in large language models as a method for passing relative position information of tokens to the ML model for training [29,31,34]. Graph-based learning has also developed similar methods for passing graph edge information alongside node information [46,47,49].

For this model, we utilize the Laplacian Position Encoding (LPE) first proposed by Dwivedi, et al. for use in ML transformers [46]. LPE calculates the eigenvectors of the weighted Laplacian matrix, constructed from a given graph, embeds the values, and adds the layer element-wise to the graph node representation.

In our model, the nodal information of the GBN dual graph consists of the state and action tensors, which each encode grain (node) data, and the GB (edge) information. This is calculated from the selected GB constitutive model for all grain neighbors after every decision step. A Laplacian can be calculated similar to Eq. (3) for the GBN puzzle, which

takes the nodes as the grains (cubes) and edges as the GB properties (connections), which can then be used to calculate eigenvectors and eigenvalues corresponding to the current state of the microstructure. While the absolute values of GB properties are vastly different between the Linear and Borisov/BRK models, this step acts to normalize the relative property connections to comparable, normalized eigenvectors.

Since the number of eigenvectors is equal to the number of nodes (grains) in a graph, the most that can be calculated for the smallest simulation is 4. To keep the amount of information constant for training, we take the first 4 eigenvectors, sorted by increasing eigenvalue, of each state as the position encoding. After embedding through a Linear layer, the shape of the resulting tensor is $[B \times 1 \times n_{pad} \times h]$, which is calculated for every t and added to the corresponding state and action tensor.

An additional position encoding is made for the time step associated with each decision. An encoding vector is made for each token (individual state, return, and action inputs), containing the time step in which the token was made. The shape of the tensor is $[B \times T]$, where T is the total number of tokens (state, return, and action for all t) in an input, and the value stored is the timestep the token appeared in. A maximum value of 810 was set for timesteps, though training data rarely reached that many decisions. After embedding with an Embedding layer, the shape of the Time Encoding tensor is $[B \times T \times h]$. This is also added to all tokens in the input layer.

3.4.4. The transformer layer

After inputs have been embedded and encodings added, the tensor is fed into a Causal Transformer [31]. The purpose of the transformer layer is to create a "soft" dictionary, where the inputs and their correlations (the query), match a known state (the key), which further has a known solution (the value) [31]. The "soft" nature of the lookup means that there does not need to be an exact match for each state (query and key) and output (value), but general trends and meanings are still captured [31].

A Causal Transformer further enforces the logic of sequential decisions by masking future tokens away from past tokens. This disallows tokens (inputs) from affecting training for future steps, while allowing future tokens to have "memory" of past tokens [30,31]. The depth of this memory is t, the number of time steps included in the input layer. This feature allows the transformer to model possible human "planning", where better property improvements are made using a sequence of decisions, rather than independent individual steps. The Causal Transformer consists of three major sections: the Block Causal Mask; the Query, Key, and Value layers; and the feed-forward layer.

3.4.5. The block causal mask

A traditional causal mask is a lower triangular matrix, which represents the sequential dependence of time series information, where the future tokens can remember the past, but the past tokens cannot know the future. The Block Causal Mask is an improvement to the information representation implemented in the Multi-Game Decision Transformer [30] that allows concurrent, but separate, pieces of information to attend to each other, unlike a traditional causal mask. This is constructed by creating a lower triangular matrix of size $[B \times T \times h]$ and value of one, and then adding a "block" of $[N \times N]$ ones, where N is the number of simultaneous observations, starting at each diagonal entry where the observation tokens begin [30].

In the Multi-Game Transformer, only the observations (sections of screen pixels) were given this consideration [30], while the GBN materials design game requires more. As all grains are considered their own agent, each grain has an action assigned during a time step, and should be allowed to "see" every grain take their action. Therefore, an additional $[N \times N]$ block is added for the actions (player decision) as well as the observations (grain orientations).

3.4.6. The query, Key, and value layers, and feed-forward

All three Query, Key, and Value (QKV) layers are Linear layers of the same shape as the final input layer that are combined to form a transformer block [31]. The input layer is fed through a Linear normalization layer, and then each of the three QKV layers in parallel. The equation for the QKV relation that defines the attention (attn) is then

$$attn = \text{soft}(\text{mask}(Q \times K^{\top}/\sqrt{h}))V \tag{6}$$

where "soft" is the softmax function, "mask" is the application of the block causal mask, Q is the query layer, K is the key layer, V is the value layer, and h is the hidden size. The cross product is only for the final two dimensions of each layer $[T \times h]$, as B the batch variable is only for parallel computations. The result is then projected back into the correct dimensionality by a Linear layer.

The resulting outputs ("logits" in ML terminology) are fed through a simple multi-layer perceptron (MLP) with a GeLU activation [67]. During training there are also dropout layers added after the QKV block and the MLP block.

3.4.7. Output layer

The logits at this step have identical shape to the inputs, with interleaved state, return, and actions. The final layer is what creates the correlation between previous states and future returns and actions. Each set of logits corresponding to the original states are fed to a Linear layer, $[h \times 5]$. The resulting output is the predicted action. Unlike other ML models, since there is a one-to-one matching of states to actions we can use the entire state information to predict future actions, instead of a single output like previous models [30,31].

For the returns, however, only the final output of each state is fed to a Linear layer, $[h \times 1]$, to predict the next return. By using the state information in this way, the next action and return become a response to the observed state.

3.4.8. Hyperparameters and training

For training and evaluating the model we set the batch size $B = 16$, hidden size $h = 1024$, and use a time step window of 4. This window sets the total tokens as $T = 404$. For training we set the optimizer as AdamW [68], the learning rate at 10^{-7}, and the decay at 10^{-6}. The low learning rate was chosen to avoid overfitting to the data, which will negatively affect the performance, but is a necessary trade-off for the desired generalizability in the model. We use 1000 warmup steps and run each training for 10 000 steps.

The training set contained 897 player trajectories consisting of between 5 and 810 decisions each. The window of 4 decisions were taken at random from a trajectory for each step of the training, following similar training methodology to [30].

A major benefit of this method is the ability to train on all data points, regardless of the quality of the training data. Since the transformer acts as a "soft" dictionary, all data contributes to knowing not only what are optimal optimization steps, but also what are non-optimal steps to be avoided.

The loss function which is minimized during training was set as the sum of the cross entropy of predicted vs. actual actions (due to one-hot encoding) and the Euclidean distance between predicted vs. actual returns. Accuracy was defined as the mean of the percent of correct actions chosen and the relative error between returns.

Validation of the model was calculated on a sequence basis during training, which was measured by the accuracy of the whole 4 time-step prediction.

As the training methodology samples the training data using random number generation, there will be some training stochasticity effects on model inference. Therefore, we will train and evaluate the ML model 10 times to evaluate the resulting variability.

3.4.9. Evaluation

The ML model was evaluated by holding 4 meshed microstructures, and the associated player trajectories, out of the training set. The microstructures had 10, 15, 25, and 30 grains respectively. Prediction was run for 810 decision steps so as to remain within the training data available. Chosen actions and total return (i.e. the game score, which was simply the value of D_{eff}) were saved for each time step.

At prediction time, the ML model only provides the action type to be applied and the selected grain to apply the action to. Algorithms are available from the video game implementation for actually applying 4 of the 5 actions to the microstructure. However, the manual rotation action must be generated separately at prediction time. Rather than training the ML model to predict exactly what manual rotation would be taken for a given step, we instead model it.

We model the manual movement of players as a local gradient ascent that maximized the properties of the connections to the currently selected grain. We justify this decision from observing player movement, where we observed that players tended to follow a rough gradient ascent in their decisions, though sometimes they overshot a local maximum. Therefore, we believe modeling the local movement as a simple gradient ascent on the local property to be an appropriate estimate of the manual motion.

The trained model is given the current state, and any previous states and actions up to the time-step window of 4, which gives the logits for the actions and the expected return. The action logits' output are the probability of choosing a given action out of the 5 available for each grain in the current state. A single action is chosen by maximum probability to be applied to the GBN simulation. The GBN orientations, boundary properties, and position encodings are then updated and used as the next state input.

Because not all player inputs used in training were optimal optimization trajectories (all data were used, not just good optimization trajectories), without additional information the predictive performance would be limited. However, as stated previously, a strength of the transformer model is the dictionary structure of the learning.

Therefore, at prediction time we add a flat bias of 0.1 (10% of maximum return) to the expected return output, and re-predict the same time step. This bias encourages the ML model to search for more optimal solutions among the training that can reach the biased score in a single step, attempting to emulate an expert (more optimal) player trajectory at prediction time [30].

The bias of 0.1 would then imply that, in the ideal scenario, the ML model would reach the maximum return in 10 decision steps. This will most likely not be the case, especially with larger grain numbers. However, the distribution of step counts for top performing players (95th percentile of all players by max return) achieving their max return is heavily skewed towards 10 steps or less in the training set. Therefore, we feel the selected bias is appropriate for attempting to generate expert trajectories.

3.5. Quantifying and comparing optimization methods

To compare the three optimization methods (SA, Player, ML) we use the same comparisons as previous work: solution quality and solution efficiency [25]. Solution quality is given by the normalized Return (the achieved material property value, D_{eff}, divided by the hypothetical upper bound value, as described in Section 3.4.1). A value of 1 represents a perfect solution, while 0 represents the worst possible. This is the same as the return input to the ML model.

For comparing the ML model against players and SA, we will consider both the best and median performing models across the 10 trained model replicates. The best ML model will be compared to the best player or the best SA, and the median performing ML model will be compared against the median player or the median SA.

Each trained model is applied to each of the 4 evaluation microstructures. The performance of each model replicate is defined to be the *smallest* maximum return across the set of 4 evaluation microstructures. That is, for a given model replicate, we consider the highest return achieved across time steps for each evaluation microstructure; the smallest of the 4 resulting values is defined to be the performance of that model replicate. This represents a worst-case measure of performance across all of the evaluation microstructures. This definition was chosen to enforce the idea that a trained model should perform well on all microstructures, not just a single one that skews the average high. Thus, the "best" and "median" ML models refer, respectively, to the replicates which exhibit the best and median performance using this definition.

To compare solution quality, we will use the following definition

$$\Delta\text{Return}_X = \text{Return}_{ML} - \text{Return}_X \tag{7}$$

where X is either $Player$ or SA. Note that $\Delta\text{Return}_X > 0$ implies the ML model achieved a higher return and $\Delta\text{Return}_X < 0$ implies the other method achieved a higher return.

The solution efficiency is measured by the number of decisions taken to achieve a fixed solution quality for the first time [25], which we choose to be the smaller of the two maximum returns of the methods being compared, according to

$$\Delta\text{Steps}_X = \text{Steps}_X - \text{Steps}_{ML} \tag{8}$$

Note that the order of arguments in Eq. (8) is reversed compared to Eq. (7). This is intentional so that positive values always indicate better ML performance (in terms of solution quality when $\Delta\text{Return}_X > 0$ or in terms of efficiency when $\Delta\text{Steps}_X > 0$).

We also test the generalizability of the ML model to other constitutive GB structure–property models that it was not trained on (the Borisov/BRK model). We will do this by repeating the evaluation on the 4 microstructures, but replacing the Linear model with the Borisov/BRK model during evaluation. No additional training nor changes to ML model inputs are done to accomplish this change.

For each of the 4 evaluation microstructures, we will compare the difference between the solution quality and solution efficiency of the ML model vs. the players, and the ML model vs. SA. However, since the Borisov/BRK model does not have any player trajectories, we will only compare the ML model vs. SA.

4. Results

4.1. Training and validation

The ML model was successfully trained and evaluated using the methods described above. The loss and accuracy values during training are shown in Fig. 4. Due to the low learning rate, the convergence of the training had some variability, but generally converged to a good accuracy. Convergence occurred at ~5000 training steps. Values for the sequence loss and accuracy, as well as the standard deviation at convergence, for the best and median trained models are shown in Table 1.

Table 1
Validation results of the training at convergence, showing the mean loss and accuracy (parenthetical values give the respective standard deviations). Note the strong stability between the best and median models.

Validation	Best model	Median model
Loss	1.77 (0.16)	1.77 (0.17)
Accuracy	0.84 (0.019)	0.84 (0.019)

4.2. Evaluation

The time history of the normalized returns resulting from application of the 3 optimization strategies (players, SA, and the ML model) to each of the 4 evaluation microstructures are shown in Fig. 5. Both the best and median performing trajectories (by return) are shown for each method.

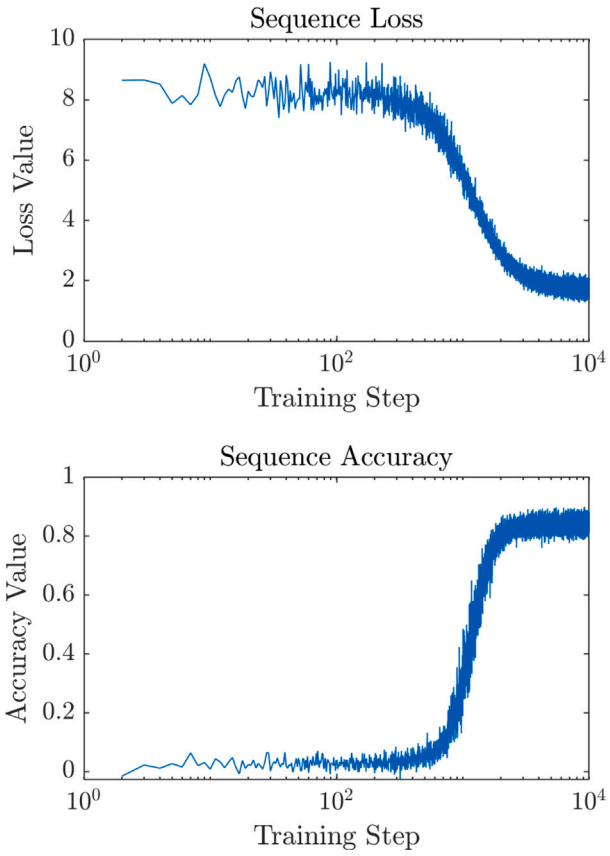

Fig. 4. Loss and accuracy values during training of the best model. Note the good convergence of values to a high accuracy.

Table 2
Performance of the best (and median, shown parenthetically) ML model compared to the best (median) players using the linear model as the constitutive relation for each structure of interest. $\Delta\text{Return}_{Player} = \text{Return}_{ML} - \text{Return}_{Player}$ (so negative values indicate higher player returns). $\Delta\text{Steps}_{Player} = \text{Steps}_{Player} - \text{Steps}_{ML}$ (so negative values indicate fewer steps were taken by the players to achieve an equivalent return).

n_{Grains}	Highest return	$\Delta\text{Return}_{Player}$	$\Delta\text{Steps}_{Player}$
10	0.810 (0.771)	−0.144 (−0.082)	−14 (−68)
15	0.904 (0.811)	0.037 (−0.022)	−82 (−507)
25	0.832 (0.736)	−0.030 (−0.079)	−329 (−628)
30	0.815 (0.771)	−0.073 (−0.037)	−465 (−736)

Table 3
Performance of the best (and median, shown parenthetically) ML model compared to the best (median) SA using the linear model as the constitutive relation for each structure of interest. $\Delta\text{Return}_{SA} = \text{Return}_{ML} - \text{Return}_{SA}$ (so negative values indicate higher SA returns). $\Delta\text{Steps}_{SA} = \text{Steps}_{SA} - \text{Steps}_{ML}$ (so negative values indicate fewer steps were taken by SA to achieve an equivalent return). The highest return columns in Tables 2 and 3 are identical because both tables consider the same ML model results.

n_{Grains}	Highest return	ΔReturn_{SA}	ΔSteps_{SA}
10	0.810 (0.771)	−0.144 (−0.168)	126 (−26)
15	0.904 (0.811)	−0.018 (−0.107)	4861 (266)
25	0.832 (0.736)	−0.056 (−0.152)	2492 (−596)
30	0.815 (0.771)	−0.093 (−0.128)	1427 (−86)

4.2.1. ML model vs. Players

As intended, the ML model performance was comparable to the player performance. However, as reported in Table 2, the players achieved slightly higher returns (on average +6.6% compared to the best ML model, and +6.7% compared to the median ML model), with the exception of the 15 grain return, for which the best ML model achieved a higher return. The players also required a smaller number of steps to achieve an equivalent return (the maximum ML return or the maximum player return, whichever was lower)—on average 223 fewer steps compared to the best ML model and 485 fewer steps compared to the median ML model. It is also worth noting, that while there is no obvious trend in $\Delta\text{Return}_{Player}$ with n_{Grains}, there does appear to be a trend of decreasing efficiency (more negative values of $\Delta\text{Steps}_{Player}$) with increasing problem size (n_{Grains}). Note also that while the efficiency of the ML model is lower than that of the players, the rise in returns have similar slopes (see Fig. 5), meaning that the ML model can increase properties at a comparable rate to the players, though sometimes this only occurs after an initial delay. These delays and lower returns could be mitigated by further tuning hyperparameters and the bias, but this would be at the risk of potential overfitting, and therefore sacrificing generalizability.

Table 4
Performance of the best (and median, shown parenthetically) ML model compared to the best (median) SA using the Borisov/BRK model as the constitutive relation for each structure of interest. $\Delta\text{Return}_{SA} = \text{Return}_{ML} - \text{Return}_{SA}$ (so negative values indicate higher SA returns). $\Delta\text{Steps}_{SA} = \text{Steps}_{SA} - \text{Steps}_{ML}$ (so negative values indicate fewer steps were taken by SA to achieve an equivalent return).

n_{Grains}	Highest return	ΔReturn_{SA}	ΔSteps_{SA}
10	0.740 (0.698)	−0.046 (−0.088)	3731 (641)
15	0.545 (0.402)	−0.234 (−0.377)	−411 (−774)
25	0.673 (0.512)	−0.063 (−0.104)	3838 (−753)
30	0.728 (0.710)	−0.047 (−0.126)	16 534 (13 868)

4.2.2. ML model vs. SA

When comparing the ML model to SA (see Table 3), we find again that, in absolute terms, the solution quality achieved by SA was slightly higher than the ML model by an amount similar to the players (i.e. the values of ΔReturn_{SA} in Table 3 are similar in magnitude to the values of $\Delta\text{Return}_{Player}$ in Table 2). However, it should be noted that SA was given a much longer time to run (the maximum number of steps allowed for SA was far greater than for players or the ML model). In contrast to the player comparison, the efficiency of the best ML model was much higher than SA (on average the best ML model required 2226 fewer steps than SA to achieve an equivalent performance). Given that the players greatly outperform SA in terms of efficiency, and the ML model is intended to emulate the players, this is one of the desired results.

5. Discussion

There are a number of encouraging observations about the ML model evaluation results. The first is the good solution quality match to the player data. Despite sparse training data, especially for the larger structures (see Fig. 6), the ML model was capable of generating quality solutions. Second, the best ML model was capable of doing so at comparable efficiency to players, and, consequently, much greater efficiency than SA.

With these promising results, we now turn our attention to questions of generalizability of the ML model. First, as explained in Section 3.1.1, we investigate whether the ML model trained on data that employed one constitutive GB structure–property model (the Linear model) can be used for optimization of microstructures with a different constitutive model applied (the Borisov/BRK model), without any additional data nor any retraining.

5.1. Generalization: Constitutive models

As described at the end of Section 3.5, to test generalizability of the ML model to optimization using the Borisov/BRK constitutive GB structure–property model (which it was *not* trained on), we simply

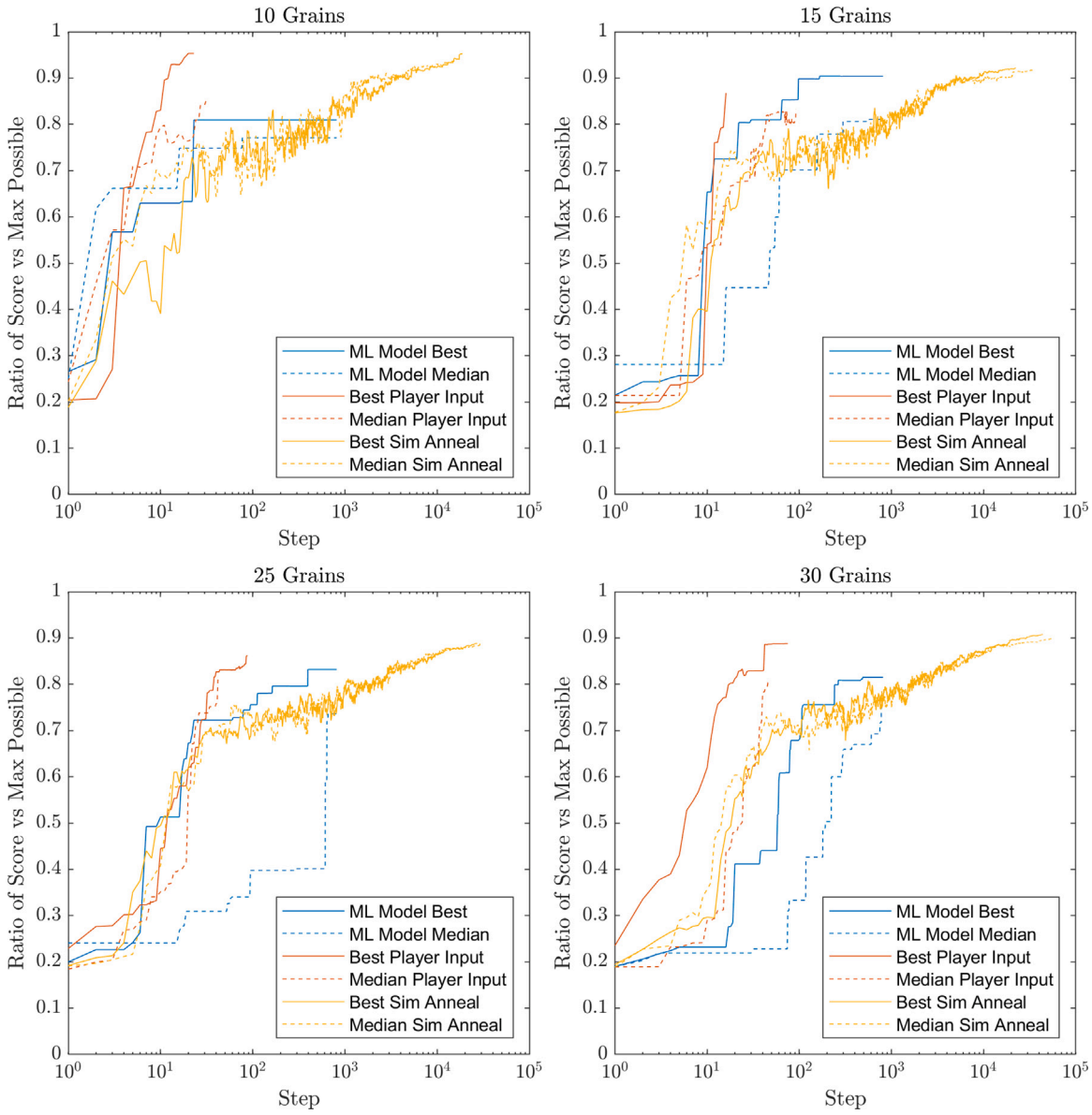

Fig. 5. Time history of returns for each method (ML model, players, and SA) using the Linear constitutive model on each of the 4 evaluation microstructures. The best and median performance (by solution quality) is shown for each method.

repeat the evaluation of the ML model for the 4 evaluation microstructures, replacing the Linear constitutive model with the Borisov/BRK model for GB property assignment. No additional training was performed. The time history of the normalized returns resulting from application of the ML model and SA (there is no player data for the Borisov/BRK model) to each of the 4 evaluation microstructures are shown in Fig. 7.

At evaluation for the Borisov/BRK model, the overall returns were lower compared to when the Linear model was used, for all 4 microstructures for both ML and SA (see Fig. 7 and Table 4). However, as reported in Table 4, the relative performance of the ML model compared to SA (ΔSteps_{SA}) remained comparable to the results observed under the Linear constitutive model, in spite of not having been trained on the Borisov/BRK model (compare Tables 3 and 4).

One notable difference, however, is that the efficiency of the ML model relative to SA is notably *improved*. The best ML model required, on average, 5923 fewer steps than SA to achieve the same return (the maximum ML return or the maximum SA return, whichever was lower), and the median ML model required, on average, 3245 fewer steps than SA.

5.1.1. Generalization: Problem size

The number of grains in both the training and evaluation microstructures are small compared to macroscopic polycrystals. However, one does not interrogate the entirety of macroscopic polycrystals during microstructure analysis. Rather, representative volume elements (RVEs) or sets of statistical volume elements (SVEs) are studied. An RVE is the smallest material volume whose properties (or structural characteristics) reflect those of the macroscopic material. A set of SVEs consists of smaller material volumes whose set-averaged properties (or structural characteristics) match those of the macroscopic material (thus for an SVE set to be fully defined both the size of each element *and* the cardinality of the set must both be specified). Just as in microstructure *analysis*, microstructure *design* can also be performed in the context of an RVE or an SVE set.

Critchfield, et al. [7] studied RVE and SVE sizes for GBNs. The RVE size for GBNs was found to be about 200 grains [7], which is approximately 10× the value of n_{Grains} in the current training data. Fortunately, the same information can be captured by using SVEs, where 12 samples of 10 grains, or 3 samples of 50 grains, can give the same information assuming 10% allowable error [7].

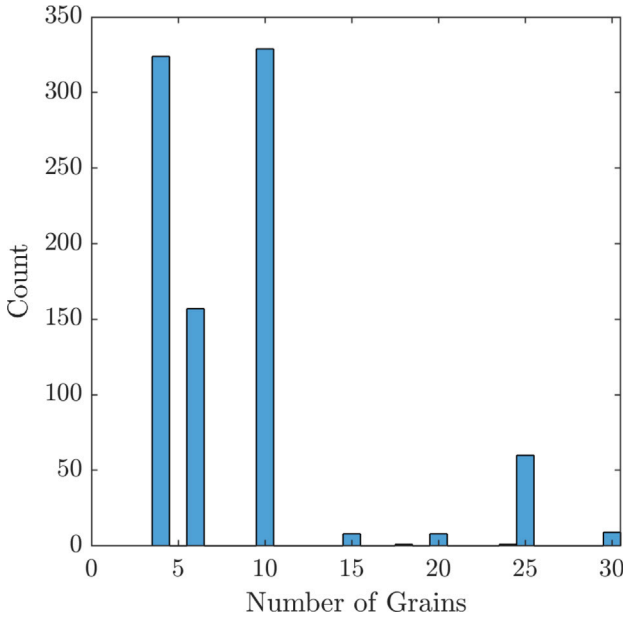

Fig. 6. Distribution of problem sizes (n_{Grains}) in the training data for the ML model.

These SVE sizes are consistent with the training data presented here. Therefore, one possible way to perform design using the ML model, could be to run multiple optimizations and then use the resulting set of microstructures as an SVE set. Thus, the success of the ML model for microstructure optimization, particularly in the context of the Borisov/BRK constitutive model, suggests the potential for solving practical GBN design problems when SVEs are employed.

However, we are also interested in investigating whether the ML model can generalize to solve design problems for microstructure sizes that it has not seen in its training data. To get at this question we first consider the distribution of n_{Grains} in the training data.

Due to the video game nature of the training data collection, the collected trajectories are heavily skewed towards puzzles presented earlier in the video game experience, which had fewer grains. Out of the 897 collected trajectories, only 9 trajectories were for grain numbers equal to 30, and 69 trajectories for grain number 25 or higher (see Fig. 6). Thus the only evaluation microstructure whose size had an appreciable amount of corresponding training data was the 10 grain microstructure. However, in spite of the paucity of corresponding training data, the returns for the 15, 25, and 30 grain microstructures under the Linear constitutive model all *exceeded* that of the 10 grain microstructure (see Table 3). For the Borisov/BRK constitutive model the 30 grain microstructure achieved the second highest return, and the highest efficiency gain relative to SA (see Table 4).

The goal of the position encoding, and separating actions to each individual grain, was to enable the ML model to learn correlations between GBN structure and effective properties, that would allow for generalization to microstructure sizes not included in the training data. That is, the hope was that by doing so the ML model would "understand" the relationship between GBN structure and properties in some kind of generalizable way. The success of the ML model for the 15, 25, and 30 grain evaluation microstructures, for which very little training data existed, provides some evidence for this kind of learning and suggests the potential of such size generalizability.

To investigate this further, we performed an experiment in which we retrained the same ML model for the Linear constitutive model player data, but removed subsets of training data. We then evaluated the resulting ML models using the Borisov/BRK constitutive GB structure–property model. This is a particularly challenging test in that it involves a systematic reduction of the training data *and* testing against a GB structure–property model that the ML model was not trained on.

The first retraining removed all training data for grain numbers less than 10, and a second retraining removed all grain numbers greater than 10. This removed 481 trajectories (53%) from our training set when removing the small grain number trajectories, and 87 trajectories (9%) when removing the large trajectories. After performing the stated retraining, the model which removed the small n_{Grains} trajectories converged to a sequence accuracy of 0.908 (SD = 0.007). This is noticeably better performance than before, but could be attributed to overfitting to a smaller dataset (more than half the training data was removed). However, when the trained model was evaluated using the Borisov/BRK constitutive model on a 6 grain sample (fewer grains than any of the supplied training data), we see that the performance is comparable to the model trained on the whole dataset when evaluated on the 10 grain evaluation microstructure (compare Figs. 8 to 7 and Table 4).

When removing the large n_{Grains} trajectories, the training converged to a sequence accuracy of 0.832 (SD = 0.019), which is close to the accuracy of the fully trained model. This can be expected because very little data is removed in this different training case. When this model is applied to the 30 grain sample (much larger than the training set) using the Borisov/BRK model it compares very closely to the previously trained model (compare Figs. 9 to 7 and Table 4).

Thus we find that the ML model is successful and shows comparable performance even when applied to evaluation microstructures whose size falls outside of the range of the training data.

5.2. Interpretability of the transformer

As a final discussion point, we consider the potential for using the transformer attention scores to interpret/understand the ML model's chosen actions. The attention scores, $Q \times K^{\top}$ from Eq. (6), provided by the transformer layer in the model have been used previously to visualize what state inputs the transformer focuses on to choose a given action. The authors of the original Decision Transformer [32] and those of the Multi-Game Transformer [30] claim this attention matrix can visualize where the model is "looking" when making decisions on a step by step basis.

An example of the attention scores for the GBN ML model for a given state step and the chosen action can be seen in the heatmap shown in Fig. 10. Each row represents the attention scores of the corresponding grain (agent) "looking" at itself and the other grains in the simulation. Fig. 10 represents step 51 of this optimization trajectory (where a large property increase resulted). At this step, the ML model chose to apply a manual rotation to grain 4 (corresponding to row 4 of the attention heatmap).

It is interesting to note that grains 2 and 9 have the highest attention scores (where the transformer is "looking"), and grain 2 is not a nearest neighbor to the selected grain. The fact that non-nearest neighbors to the selected grain have the highest attention scores could reflect that optimal GBN properties are dependent on intermediate to long range connectivity effects, and suggest that the ML model has learned something about such longer-range relationships for this specific time step. Alternatively, non-nearest neighbor importance may instead reflect optimization strategies learned from the human player behavior, for example a multi-step *sequence* of decisions that improve the performance: a particular decision at this time step may not have an immediate impact, but it may anticipate and enable a dependent future decision that does improve the performance (similar to preparatory moves in chess that set the stage for and enable a future action).

We note, however, that interpretation of attention scores is still debated in the ML community as the attention shown is simply the cross product of the final two dimensions, T and h, of two Linear layers, and robust definitions and understanding are still being developed [33,34].

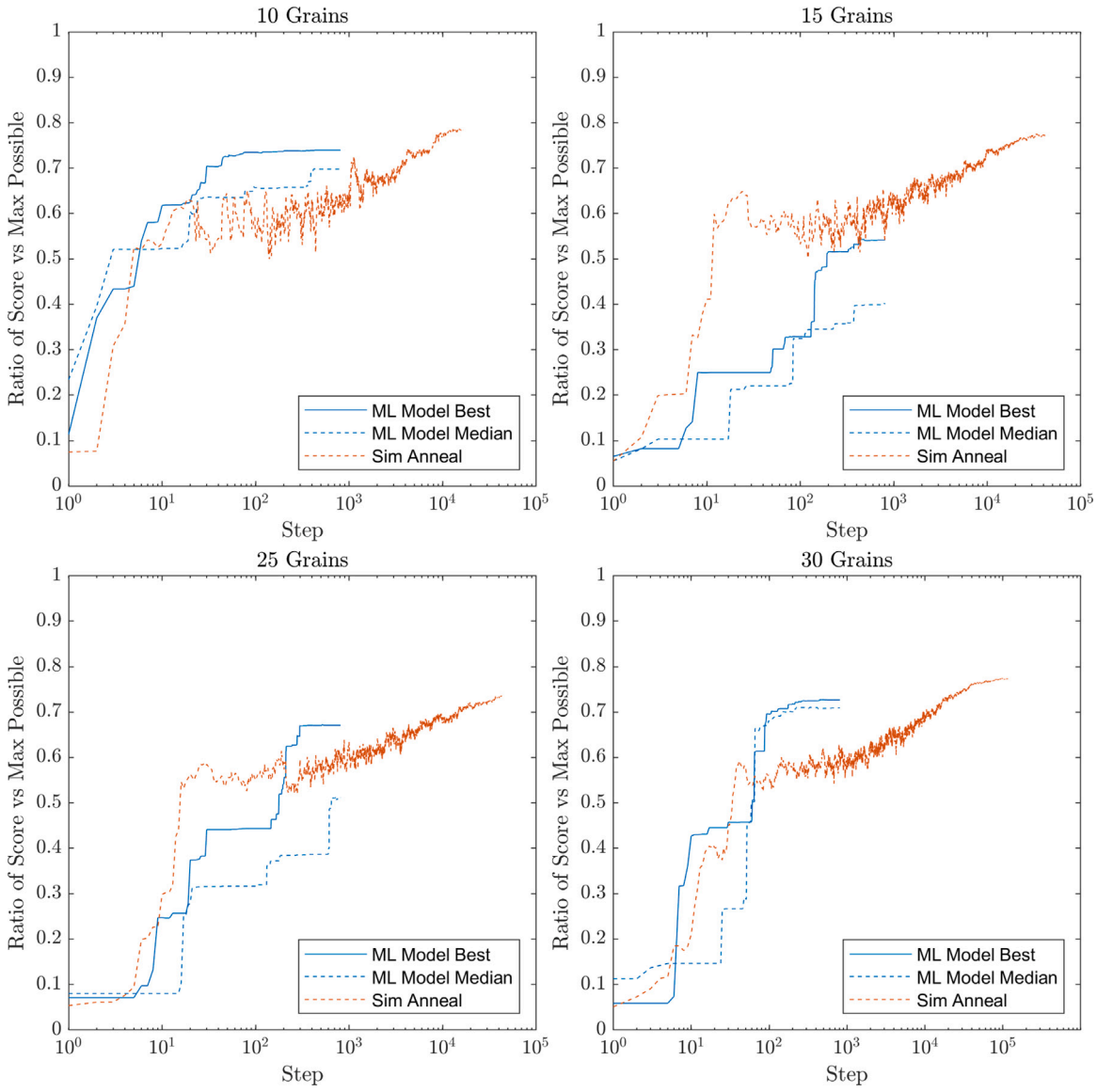

Fig. 7. Time history of returns for the ML model and SA using the Borisov/BRK constitutive model on each of the 4 evaluation microstructures. The best and median performance (by solutions quality) is shown for the ML model.

6. Conclusions

Optimization of GBNs is difficult due to their high-dimensional state space, but would enable design of materials with enhanced properties in multiple application areas. Prior work [25] showed that leveraging human spatial reasoning and intuited dimensionality reducing heuristics via a video game was a viable route for obtaining optimization pathways, but such data are expensive and slow to obtain.

In the present work, we demonstrated how such human optimization trajectories for GBNs can be used to train a Decision Transformer ML model to perform microstructure design. The resulting ML model uses all crystallographic information and long range connectivity to generate microstructures with optimized macroscopic properties for a given constitutive GB structure–property model. This implementation of the Decision Transformer expands the capabilities of the model to allow multiple concurrent actions during a decision step, rather than a single action per set of observations like previous implementations. This allowed the model to act on a variable amount of grains with a set of actions, and used the same position encoding as the observation space.

When compared to simulated annealing (a popular global optimization algorithm), we found that the ML model obtained solutions that were of comparable quality (on average 92% as optimal as the SA solutions). However, we found that the ML model was far more efficient, requiring three orders of magnitude fewer iterations to do so.

We found that the ML model was capable of generalizing to constitutive structure–property models on which it was not trained. In particular, we trained the ML model on human player data from design problems that employed a simple (and computationally inexpensive) GB constitutive structure–property model. We then used the trained model to solve design problems for a high-fidelity (and computationally expensive) GB structure–property model *without* any additional training. We found that the ML model showed similar and in some cases even better performance (relative returns and efficiency gains over SA). This capability is particularly useful when training data for the constitutive structure–property model of interest is unavailable, but one does have access to training data for a different (e.g. simpler) structure–property model.

We demonstrated that the ML model can solve problem sizes relevant to real microstructure design applications, particularly when statistical volume elements (SVEs) are employed. We also found evidence

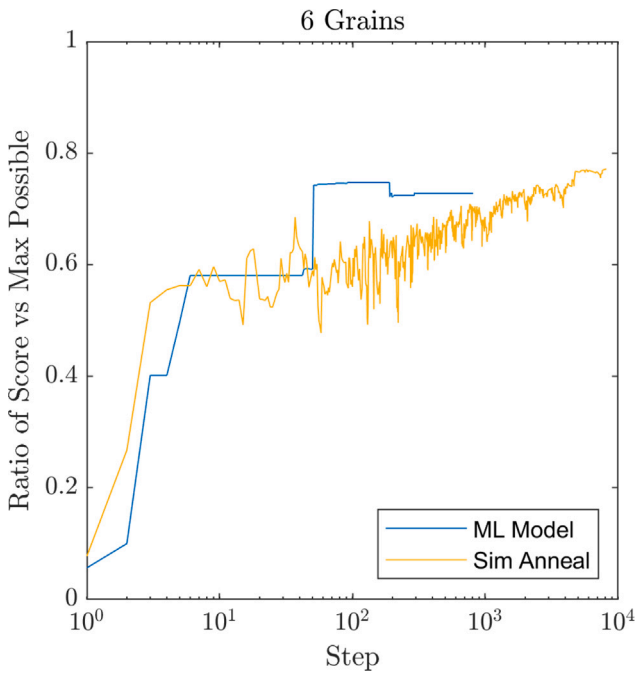

Fig. 8. Trajectories of a model that is only trained on player data containing microstructures with 10 or more grains. Note the comparable performance to the 10 grain problem in Fig. 7, which used all of the training data.

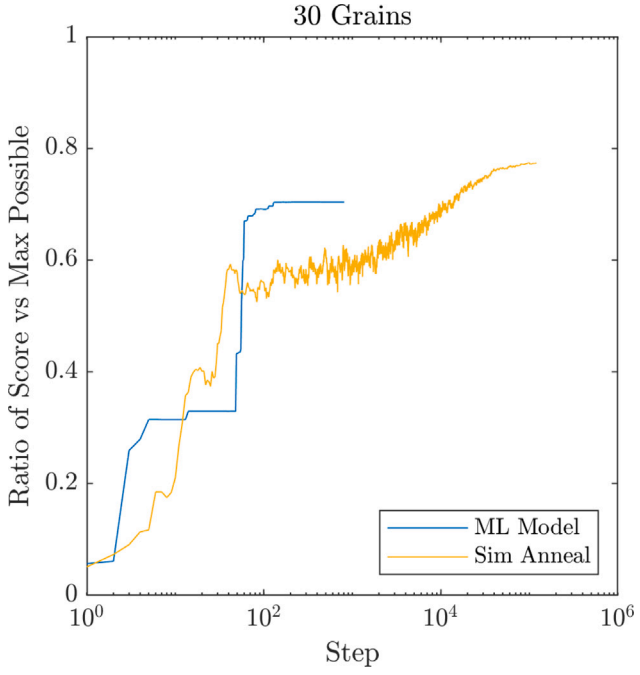

Fig. 9. Trajectories of a model that is only trained on player data containing microstructures with 10 or fewer grains. Note the similar performance to the 30 grain case in Fig. 7 even though no training data was given for this grain number.

for generalizability to problem sizes outside of the range considered in training data, which may allow for solution of larger design problems.

Finally, we gave a brief illustration of how the underlying attention scores can be visualized, which may possibly lead to interpretation of the optimization strategies employed by the ML model.

Based on these results, we believe Decision Transformer based models may allow for efficient design of GBNs, and perhaps materials design

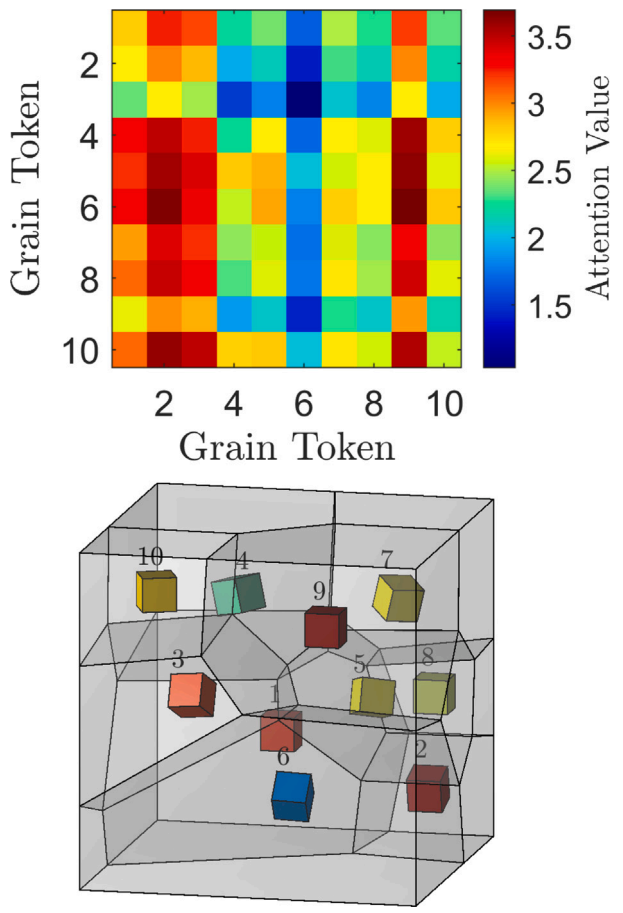

Fig. 10. Visualization of the attention block of the state at time step 51, as well as a projection of the attention scores of the chosen grain (grain 4) onto the microstructure.

problems generally. As algorithms, hardware, and understanding of configuration spaces improve, we hope these data sets and models are also useful stepping stones to understanding human decision making and interpreting high-dimensional information.

CRediT authorship contribution statement

Christopher W. Adair: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Oliver K. Johnson:** Writing – review & editing, Visualization, Validation, Supervision, Resources, Project administration, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Oliver K. Johnson reports financial support was provided by National Science Foundation. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The material presented here is based upon work supported by the National Science Foundation, United States under Grant No. DMR-1654700.

Data availability

Both data and model code are available in the linked repository

A-Decision-Transformer-Approach-to-Grain-Boundary-Network-Optimization (Original data) (GitHub)

References

[1] D. jun Liu, G. Tian, G. feng Jin, W. Zhang, Y. hong Zhang, Characterization of localized corrosion pathways in 2195-T8 Al–Li alloys exposed to acidic solution, Def. Technol. 25 (2023) 152–165, http://dx.doi.org/10.1016/j.dt.2022.05.004.

[2] J. Sayet, B.O. Hoch, A. Oudriss, J. Bouhattate, X. Feaugas, Multi-scale approach to hydrogen diffusion in FCC polycrystalline structure with binary classification of grain boundaries in continuum model, Mater. Today Commun. 34 (January 2020) (2023) 105021, http://dx.doi.org/10.1016/j.mtcomm.2022.105021.

[3] S. Bechtle, M. Kumar, B.P. Somerday, M.E. Launey, R.O. Ritchie, Grain-boundary engineering markedly reduces susceptibility to intergranular hydrogen embrittlement in metallic materials, Acta Mater. 57 (14) (2009) 4148–4157, http://dx.doi.org/10.1016/j.actamat.2009.05.012.

[4] B.D. Snow, S.G. Baird, C. Kurniawan, D.T. Fullwood, E.R. Homer, O.K. Johnson, Grain boundary structure-property model inference using polycrystals: The underdetermined case, Acta Mater. 209 (2021) 116769, http://dx.doi.org/10.1016/j.actamat.2021.116769.

[5] O.K. Johnson, J.M. Lund, T.R. Critchfield, Spectral graph theory for characterization and homogenization of grain boundary networks, Acta Mater. 146 (2018) 42–54, http://dx.doi.org/10.1016/j.actamat.2017.11.054.

[6] W. Feng, Z. Wang, Q. Sun, Y. He, Y. Sun, Effect of thermomechanical processing via rotary swaging on grain boundary character distribution and intergranular corrosion in 304 austenitic stainless steel, J. Mater. Res. Technol. 19 (2022) 2470–2482, http://dx.doi.org/10.1016/j.jmrt.2022.06.032.

[7] T.R. Critchfield, O.K. Johnson, Representative and statistical volume elements for grain boundary networks: A stereological approach, Acta Mater. 188 (2020) 166–180, http://dx.doi.org/10.1016/j.actamat.2019.12.029.

[8] M. Kamaya, A procedure for estimating Young's modulus of textured polycrystalline materials, Int. J. Solids Struct. 46 (13) (2009) 2642–2649, http://dx.doi.org/10.1016/j.ijsolstr.2009.02.013.

[9] X.-L. Peng, B.-X. Xu, Unraveling impacts of polycrystalline microstructures on ionic conductivity of ceramic electrolytes by computational homogenization and machine learning, J. Appl. Phys. 136 (10) (2024) 105101, http://dx.doi.org/10.1063/5.0223138, arXiv:https://pubs.aip.org/aip/jap/article-pdf/doi/10.1063/5.0223138/20150347/105101_1_5.0223138.pdf.

[10] M. Dai, M.F. Demirel, X. Liu, Y. Liang, J.-M. Hu, Graph neural network for predicting the effective properties of polycrystalline materials: A comprehensive analysis, Comput. Mater. Sci. 230 (2023) 112461, http://dx.doi.org/10.1016/j.commatsci.2023.112461, URL: https://www.sciencedirect.com/science/article/pii/S092702562300455X.

[11] A.V. Sergueeva, N.A. Mara, A.K. Mukherjee, Universal features of grain boundary networks in FCC materials, J. Mater. Sci. 40 (4) (2005) 847–852, http://dx.doi.org/10.1007/s10853-006-0697-0, URL: http://www.springerlink.com/index/10.1007/s10853-006-0697-0.

[12] T. Watanabe, S. Tsurekawa, Control of brittleness and development of desirable mechanical properties in polycrystalline systems by grain boundary engineering, Acta Mater. 47 (15) (1999) 4171–4185, http://dx.doi.org/10.1016/S1359-6454(99)00275-X.

[13] H. Zheng, X.-g. Li, R. Tran, C. Chen, M. Horton, D. Winston, K. Aslaug, S. Ping, Acta materialia grain boundary properties of elemental metals, Acta Mater. 186 (2020) 40–49, http://dx.doi.org/10.1016/j.actamat.2019.12.030.

[14] V. Tcherdyntsev, A. Rodin, The algorithm to predict the grain boundary diffusion in non-dilute metallic systems, Materials 16 (4) (2023) http://dx.doi.org/10.3390/ma16041431.

[15] S. Hao, H. Li, Effect of twin grain boundary on the diffusion of Cu in bulk β-Sn, Comput. Mater. Sci. 226 (May) (2023) 112200, http://dx.doi.org/10.1016/j.commatsci.2023.112200.

[16] A. Oudriss, J. Creus, J. Bouhattate, E. Conforto, C. Berziou, C. Savall, X. Feaugas, Grain size and grain-boundary effects on diffusion and trapping of hydrogen in pure nickel, Acta Mater. 60 (19) (2012) 6814–6828, http://dx.doi.org/10.1016/j.actamat.2012.09.004.

[17] P. Kontis, E. Chauvet, Z. Peng, J. He, A.K. da Silva, D. Raabe, C. Tassin, J.J. Blandin, S. Abed, R. Dendievel, B. Gault, G. Martin, Atomic-scale grain boundary engineering to overcome hot-cracking in additively-manufactured superalloys, Acta Mater. 177 (2019) 209–221, http://dx.doi.org/10.1016/j.actamat.2019.07.041, arXiv:1905.09537.

[18] X. Meng, Z. Liu, B. Cui, D. Qin, H. Geng, W. Cai, L. Fu, J. He, Z. Ren, J. Sui, Grain boundary engineering for achieving high thermoelectric performance in n-type skutterudites, Adv. Energy Mater. 7 (13) (2017) http://dx.doi.org/10.1002/aenm.201602582.

[19] J. Guo, Q. Lou, Y. Qiu, Z.Y. Wang, Z.H. Ge, J. Feng, J. He, Remarkably enhanced thermoelectric properties of Bi2S3 nanocomposites via modulation doping and grain boundary engineering, Appl. Surf. Sci. 520 (January) (2020) 146341, http://dx.doi.org/10.1016/j.apsusc.2020.146341.

[20] T. Fast, M. Knezevic, S.R. Kalidindi, Application of microstructure sensitive design to structural components produced from hexagonal polycrystalline metals, Comput. Mater. Sci. 43 (2) (2008) 374–383, http://dx.doi.org/10.1016/j.commatsci.2007.12.002.

[21] B.B. Zhang, Y.G. Tang, Q.S. Mei, X.Y. Li, K. Lu, Inhibiting creep in nanograined alloys with stable grain boundary networks, Science 378 (6620) (2022) 659–663, http://dx.doi.org/10.1126/science.abq7739.

[22] R. Krakow, R.J. Bennett, D.N. Johnstone, Z. Vukmanovic, W. Solano-Alvarez, S.J. Lainé, J.F. Einsle, P.A. Midgley, C.M. Rae, R. Hielscher, On three-dimensional misorientation spaces, Proc. R. Soc. A: Math. Phys. Eng. Sci. 473 (2206) (2017) http://dx.doi.org/10.1098/rspa.2017.0274.

[23] E.R. Homer, S. Patala, J.L. Priedeman, Grain boundary plane orientation fundamental zones and structure-property relationships, Sci. Rep. 5 (2015) 15476, http://dx.doi.org/10.1038/srep15476, URL: http://www.nature.com/articles/srep15476.

[24] A. Lecchini-Visintini, J. Lygeros, J. MacIejowski, Simulated annealing: Rigorous finite-time guarantees for optimization on continuous domains, in: Advances in Neural Information Processing Systems 20 - Proceedings of the 2007 Conference, 2009, arXiv:0709.2989.

[25] C.W. Adair, H. Evans, E. Beatty, D.L. Hansen, S. Holladay, O.K. Johnson, Microstructure design using a human computation game, Materialia 25 (May) (2022) 101544, http://dx.doi.org/10.1016/j.mtla.2022.101544.

[26] A.J. Quinn, B.B. Bederson, Human computation: A survey and taxonomy of a growing field, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2011, pp. 1403–1412, http://dx.doi.org/10.1145/1978942.1979148.

[27] J.H.M. Jensen, M. Gajdacz, S.Z. Ahmed, J.H. Czarkowski, C. Weidner, J. Rafner, J.J. Sørensen, K. Mølmer, J.F. Sherson, Crowdsourcing human common sense for quantum control, Phys. Rev. Res. 3 (1) (2021) 1–23, http://dx.doi.org/10.1103/PhysRevResearch.3.013057, arXiv:2004.03296.

[28] F. Khatib, F. Dimaio, S. Cooper, M. Kazmierczyk, M. Gilski, S. Krzywda, H. Zabranska, I. Pichova, J. Thompson, Z. Popović, M. Jaskolski, D. Baker, Crystal structure of a monomeric retroviral protease solved by protein folding game players, Nat. Struct. Mol. Biol. 18 (10) (2010) 1175–1177, http://dx.doi.org/10.1038/nsmb.2119.

[29] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: transformers for image recognition at scale, in: ICLR 2021 - 9th International Conference on Learning Representations, 2021, arXiv:2010.11929.

[30] K.H. Lee, O. Nachum, M. Yang, L. Lee, D. Freeman, W. Xu, S. Guadarrama, I. Fischer, E. Jang, H. Michalewski, I. Mordatch, Multi-game decision transformers, in: Advances in Neural Information Processing Systems, vol. 35, 2022, arXiv:2205.15241.

[31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, vol. 2017-Decem, 2017, pp. 5999–6009, arXiv:1706.03762.

[32] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, I. Mordatch, Decision transformer: Reinforcement learning via sequence modeling, in: Advances in Neural Information Processing Systems, vol. 18, 2021, pp. 15084–15097, arXiv:2106.01345.

[33] H. Chefer, S. Gur, L. Wolf, Transformer interpretability beyond attention visualization, in: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2021, pp. 782–791, http://dx.doi.org/10.1109/CVPR46437.2021.00084, arXiv:2012.09838.

[34] M. Bhan, N. Achache, V. Legrand, A. Blangero, N. Chesneau, Evaluating self-attention interpretability through human-grounded experimental protocol, in: Communications in Computer and Information Science, vol. 1903 CCIS, 2023, pp. 26–46, http://dx.doi.org/10.1007/978-3-031-44070-0_2, arXiv:2303.15190.

[35] S. Fu, Y. Arinicheva, C. Hüter, M. Finsterbusch, R. Spatschek, Grain boundary characterization and potential percolation of the solid electrolyte LLZO, Batteries 9 (4) (2023) 1–14, http://dx.doi.org/10.3390/batteries9040222.

[36] X. Cheng, J. Zheng, J. Lu, Y. Li, P. Yan, Y. Zhang, Realizing superior cycling stability of Ni-rich layered cathode by combination of grain boundary engineering and surface coating, Nano Energy 62 (March) (2019) 30–37, http://dx.doi.org/10.1016/j.nanoen.2019.05.021.

[37] T.P. Swiler, V. Tikare, E.A. Holm, Heterogeneous diffusion effects i n polycrystalline, Mater. Sci. Eng. A238 (1997) 85–93.

[38] C.W. Adair, O.K. Johnson, Materialia characterizing grain boundary network length features through a harmonic representation, Materialia 35 (May) (2024) 102121, http://dx.doi.org/10.1016/j.mtla.2024.102121.

[39] V.V. Bulatov, B.W. Reed, M. Kumar, Grain boundary energy function for fcc metals, Acta Mater. 65 (2014) 161–175, http://dx.doi.org/10.1016/j.actamat.2013.10.057.

[40] R. Quey, P.R. Dawson, F. Barbe, Large-scale 3D random polycrystals for the finite element method: Generation, meshing and remeshing, Comput. Methods Appl. Mech. Engrg. 200 (17–20) (2011) 1729–1745, http://dx.doi.org/10.1016/j.cma.2011.01.002.

[41] L. von Ahn, Games with a purpose, Computer 39 (6) (2006) 92–94, http://dx.doi.org/10.1109/MC.2006.196, URL: http://ieeexplore.ieee.org/document/1642623/.

[42] M. Krause, J. Smeddinck, Human computation games: A survey, Eur. Signal Process. Conf. (Eusipco) (2011) 754–758.

[43] L. von Ahn, L. Dabbish, Designing games with a purpose, Commun. ACM 51 (8) (2008) 57, http://dx.doi.org/10.1145/1378704.1378719, URL: http://portal.acm.org/citation.cfm?doid=1378704.1378719.

[44] A. Kawrykow, G. Roumanis, A. Kam, D. Kwak, C. Leung, C. Wu, E. Zarour, L. Sarmenta, M. Blanchette, J. Waldispühl, Phylo: A citizen science approach for improving multiple sequence alignment, PLoS One 7 (3) (2012) http://dx.doi.org/10.1371/journal.pone.0031362.

[45] C.W. Adair, Operation: Forge the deep, 2022, URL: https://store.steampowered.com/app/2053100/Operation_Forge_the_Deep/.

[46] V.P. Dwivedi, X. Bresson, A generalization of transformer networks to graphs, AAAI (2021) arXiv:2012.09699. URL: http://arxiv.org/abs/2012.09699.

[47] G. Mialon, D. Chen, M. Selosse, J. Mairal, GraphiT: Encoding graph structure in transformers, 2021, pp. 1–20, arXiv:2106.05667. URL: http://arxiv.org/abs/2106.05667.

[48] S. Maskey, A. Parviz, M. Thiessen, H. Stärk, Y. Sadikaj, H. Maron, Generalized Laplacian positional encoding for graph representation learning, in: NeurIPS Workshop on Symmetry and Geometry in Neural Representations, 2022, arXiv:2210.15956. URL: http://arxiv.org/abs/2210.15956.

[49] W. Park, W. Chang, D. Lee, J. Kim, S.-w. Hwang, GRPE: Relative positional encoding for graph transformer, 2022, arXiv:2201.12787. URL: http://arxiv.org/abs/2201.12787.

[50] L. Rampášek, M. Galkin, V.P. Dwivedi, A.T. Luu, G. Wolf, D. Beaini, Recipe for a general, powerful, scalable graph transformer, in: Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 1–27, arXiv:2205.12454.

[51] L.G. Harrison, Influence of dislocations on diffusion kinetics in solids with particular reference to the alkali halides, Trans. Faraday Soc. 57 (1961) 1191–1199, http://dx.doi.org/10.1039/TF9615701191.

[52] S. Dube, J. Gorimbo, M. Moyo, C.G. Okoye-Chine, X. Liu, Synthesis and application of Ni-based membranes in hydrogen separation and purification systems: A review, J. Environ. Chem. Eng. 11 (2023) http://dx.doi.org/10.1016/j.jece.2022.109194.

[53] J. Łuczak, M. Lieder, Nickel-based catalysts for electrolytic decomposition of ammonia towards hydrogen production, Adv. Colloid Interface Sci. 319 (2023) http://dx.doi.org/10.1016/j.cis.2023.102963.

[54] Z. Gao, T.S. Suzuki, S. Grasso, Y. Sakka, M.J. Reece, Highly anisotropic single crystal-like La2Ti2O7 ceramic produced by combined magnetic field alignment and templated grain growth, J. Eur. Ceram. Soc. 35 (6) (2015) 1771–1776, http://dx.doi.org/10.1016/j.jeurceramsoc.2014.12.003, URL: https://www.sciencedirect.com/science/article/pii/S0955221914006608.

[55] K.A. Sofinowski, S. Raman, X. Wang, B. Gaskey, M. Seita, Layer-wise engineering of grain orientation (LEGO) in laser powder bed fusion of stainless steel 316L, Addit. Manuf. 38 (2021) 101809, http://dx.doi.org/10.1016/j.addma.2020.101809, URL: https://www.sciencedirect.com/science/article/pii/S2214860420311817.

[56] R.W. Balluffi, S.M. Allen, W.C. Carter, Kinetics of Materials, Wiley-Interscience, 2005, http://dx.doi.org/10.1002/0471749311.

[57] L.G. Harrison, Influence of dislocations on diffusion kinetics in solids with particular reference to the alkali halides, Trans. Faraday Soc. 57 (1961) 1191–1199, http://dx.doi.org/10.1039/TF9615701191.

[58] J.F. Nye, Physical Properties of Crystals Their Representation by Tensors and Matrices by J.F. Nye, Clarendon Press, 2012.

[59] D.E. Page, K.F. Varela, O.K. Johnson, D.T. Fullwood, E.R. Homer, Measuring simulated hydrogen diffusion in symmetric tilt nickel grain boundaries and examining the relevance of the borisov relationship for individual boundary diffusion, Acta Mater. 212 (2021) http://dx.doi.org/10.1016/j.actamat.2021.116882.

[60] V. Borisov, V. Golikov, G. Scherbedinskiy, Relation between diffusion coefficients and grain boundary energy, Phys. Met. Met. 17 (1964) 881–885.

[61] P. Schweizer, A. Sharma, L. Pethö, E. Huszar, L.M. Vogl, J. Michler, X. Maeder, Atomic scale volume and grain boundary diffusion elucidated by in situ STEM, Nat. Commun. 14 (2023) http://dx.doi.org/10.1038/s41467-023-43103-7.

[62] V. Mohles, 3-D front tracking model for interfaces with anisotropic energy, Comput. Mater. Sci. 176 (2020) 109534, http://dx.doi.org/10.1016/j.commatsci.2020.109534, URL: https://linkinghub.elsevier.com/retrieve/pii/S0927025620300252.

[63] S.K. Naghibzadeh, Z. Xu, D. Kinderlehrer, R. Suter, K. Dayal, G.S. Rohrer, Impact of grain boundary energy anisotropy on grain growth, Phys. Rev. Mater. 8 (2024) http://dx.doi.org/10.1103/PhysRevMaterials.8.093403.

[64] C. Kurniawan, S. Baird, D.T. Fullwood, E.R. Homer, O.K. Johnson, Grain boundary structure–property model inference using polycrystals: The overdetermined case, J. Mater. Sci. 55 (4) (2020) 1562–1576, http://dx.doi.org/10.1007/s10853-019-04125-z.

[65] A. Dukkipati, M.N. Murty, S. Bhatnagar, Cauchy annealing schedule: An annealing schedule for Boltzmann selection scheme in evolutionary algorithms, in: Proceedings of the 2004 Congress on Evolutionary Computation, CEC2004, vol. 1, 2004, pp. 55–62, http://dx.doi.org/10.1109/cec.2004.1330837, arXiv:0408055.

[66] M. Dwarampudi, N.V.S. Reddy, Effects of padding on LSTMs and CNNs, 2019, arXiv:1903.07288. URL: http://arxiv.org/abs/1903.07288.

[67] M. Lee, Mathematical analysis and performance evaluation of the gelu activation function in deep learning, J. Math. Univ. Tokushima 2023 (2023) http://dx.doi.org/10.1155/2023/4229924.

[68] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: 7th International Conference on Learning Representations ICLR 2019, 2019, arXiv:1711.05101.